

ISM2411 — Lab Week 13

First DataFrame — Retail Sales Explorer — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 13 · Unit 4 · Data Analysis with pandas

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	5
6.1	Intro (0:00–0:04)	5
6.2	Exercise 1 — Load It (0:04–0:12, 8 min)	5
6.3	Exercise 2 — Filter by Region (0:12–0:20, 8 min)	8
6.4	Exercise 3 — Top 5 by Revenue (0:20–0:27, 7 min)	10
6.5	Exercise 4 — Two-Condition Filter (0:27–0:36, 9 min)	12
6.6	Exercise 5 — A New Column (0:36–0:44, 8 min)	14
6.7	Exercise 6 — Unique Values (0:44–0:51, 7 min)	16
6.8	Exercise 7 — Select Columns (0:51–0:57, 6 min)	18
6.9	Exercise 8 — Save (0:57–1:02, 5 min)	19
6.10	Stretch 1 & 2 Preview (1:02–1:10, as time allows)	21
7	Wrap-Up (last ~5 minutes)	22
8	Appendix A — Full Answer Key (explore.py)	22
9	Appendix B — Reproducible Synthetic Dataset (for instructor testing)	24
10	Appendix C — Extra Practice (only if the class finishes early)	25

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 13 Lab — First DataFrame: Retail Sales Explorer
Unit	Unit 4 · Data Analysis with pandas
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 12: CSV files, the general read-transform-write shape; Module 10–11: lists, dictionaries, boolean logic
Student-facing lab page	markumreed.github.io/ism2411 — week13_lab
Exercises covered	Exercises 1–8 (required) + Stretch 1/2 (as time allows)
Submission	<code>explore.py</code> (or <code>explore.ipynb</code>) via GitHub in a <code>module13/</code> folder, URL to Canvas

The lab page’s own framing is exactly right and worth repeating verbatim to the class: this is “the foundation for every subsequent lab and the capstone.” Everything from here to the end of the semester builds on the DataFrame vocabulary introduced today — `.head()`, boolean filtering, `.sort_values()`, new columns, `.unique().value_counts()`. Two ideas deserve the most protected time: the **boolean filter mental model** (Exercise 2 — a filter is a same-length column of True/False, not a search) and the **&/parentheses requirement** (Exercise 4), which is a real, common source of genuinely confusing errors if not addressed head-on.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Load a CSV into a DataFrame with `pd.read_csv()` and inspect its shape, columns, and dtypes.
2. Explain what a boolean filter (`df[df['region'] == 'South']`) actually does — construct a column of True/False, then keep only the True rows.
3. Combine two conditions with `&`, and explain why both parentheses and `&` (not `and`) are required, not stylistic.
4. Create a new column from existing ones with a single vectorized assignment, with no loop.
5. Use `.unique()` and `.value_counts()` to understand what values live in a column and how often.
6. Save a DataFrame back to CSV with `.to_csv()`.

3 Before Class — Setup Checklist

- ☐ Confirm pandas is installed on your demo machine (`import pandas as pd` runs with no error) and, if not already covered, budget two minutes for `pip install pandas` — first import pandas of the semester.
- ☐ Obtain or generate `data/retail_sales.csv` before class — if your course's actual dataset isn't yet in hand, Appendix B documents a verified, reproducible synthetic dataset generator (5,000 rows, 6 columns, matching the lab page's expected shape) you can run to produce a working file for testing and demo purposes. **Use your course's real provided dataset with students** — the specific numbers in this guide are verified against the synthetic version and are for your own preparation, not something to read aloud as if they were the real assignment's answers.
- ☐ Rehearse Exercise 4's two live error demos (& without parentheses, and instead of &) before class — both produce genuinely confusing-looking errors the first time, and being ready to explain them calmly, rather than looking surprised yourself, matters for this exercise landing well.

4 Materials Needed

- Instructor laptop + terminal + editor (or Jupyter), Python 3.10+, pandas installed
- Students: `data/retail_sales.csv`, same GitHub repo with a new `module13/` folder

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: “the tool for the rest of the semester”	4
0:04–0:12	Exercise 1 — Load it	8
0:12–0:20	Exercise 2 — Filter by region	8
0:20–0:27	Exercise 3 — Top 5 by revenue	7
0:27–0:36	Exercise 4 — Two-condition filter	9
0:36–0:44	Exercise 5 — A new column	8
0:44–0:51	Exercise 6 — Unique values	7
0:51–0:57	Exercise 7 — Select columns	6
0:57–1:02	Exercise 8 — Save	5
1:02–1:10	Stretch 1/2 preview	8
1:10–1:15	Wrap-up, reflection, submission checklist	5

Eight required exercises fill the bulk of the 75 minutes; both Stretch challenges are positioned as

previews since Stretch 1 explicitly depends on groupby, a Module 15 concept the lab page itself flags as ahead of this lab's normal sequence.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Every module in Unit 3 processed data with lists, dictionaries, and hand-written loops. Today we meet pandas — a library purpose-built for exactly this kind of tabular, spreadsheet-shaped data. A lot of what took a full loop and an accumulator in Module 12 collapses into a single line today. This is the tool you’ll use for the rest of the semester, including the capstone — get comfortable with today’s vocabulary.”

Do: Open `explore.py`, type the header:

```
# explore.py
# ISM2411 Module 13 Lab – First DataFrame: Retail Sales Explorer
```

6.2 Exercise 1 — Load It (0:04–0:12, 8 min)

Teaching goal: `pd.read_csv()` to load a file into a **DataFrame**, and four inspection methods (`.head()`, `.shape`, `.columns.tolist()`, `.dtypes`) that should become an automatic first step every time a new dataset is loaded.

Say to the class:

“One line loads an entire CSV into a DataFrame — think of a DataFrame as a spreadsheet living inside your program. Then four inspection calls that I want to become automatic muscle memory: every single time you load a new dataset, run these four before doing anything else.”

Live-code this:

```
# --- Exercise 1 ---
import pandas as pd

df = pd.read_csv('data/retail_sales.csv')
print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.dtypes)
```

Line-by-line explanation:

- `import pandas as pd` — pandas is a third-party library (not part of Python’s standard library, unlike Module 12’s `csv`) — say explicitly it needs `pip install pandas` if not already present.

as `pd` is an **alias** — a shorter name to type everywhere else in the script; `pd` specifically is such a universal convention in the Python data community that using anything else would actually confuse other readers of your code.

- `pd.read_csv('data/retail_sales.csv')` — reads the entire CSV file and returns a **DataFrame**: a two-dimensional table, with labeled columns and a numeric row index, automatically inferring each column's data type from its contents. Contrast this explicitly and briefly with Module 12: this single line replaces an entire `open(...)` as `f: reader = csv.reader(f)` block plus manual type conversion — worth stating plainly as the direct payoff of using a purpose-built library.
- `df.head()` — shows the **first 5 rows** by default — a quick sanity check that the data loaded correctly and looks like what you expect, without printing the (potentially huge) entire table.
- `df.shape` — a tuple of (rows, columns) — **not a method call**, no parentheses — say explicitly: `.shape` is an **attribute**, a piece of information the DataFrame already has stored, not something it computes fresh via a function call; contrast this with `.head()`, which *is* a method (has parentheses) since it does real work (selecting and formatting rows) each time it's called.
- `df.columns.tolist()` — `.columns` alone gives back a specialized pandas object listing column names; `.tolist()` converts that into an ordinary Python list, easier to read and work with directly.
- `df.dtypes` — shows the inferred data type of every column: `int64` for whole numbers, `float64` for decimals, `object` for text (pandas' catch-all type for strings) — worth flagging `object` explicitly as pandas' term for “text,” since it doesn't obviously read that way to a beginner encountering it for the first time.

Run it. Expected output (verified against a 5,000-row, 6-column synthetic dataset — see Appendix B; exact values will differ with your course's real data, but the *shape* of the output — five rows, a (5000, 6)-style tuple, six named columns, a dtype per column — should look structurally identical):

```
order_id    date region  product  quantity  revenue
0    100001  2024-07-01  East  Gadget C         2    19.98
1    100002  2024-05-27  South Widget B        11   549.89
2    100003  2024-03-23  West  Gadget D        12   179.88
3    100004  2024-01-17  East  Gadget D        15   749.85
4    100005  2024-01-31  North Widget B        12   299.88
(5000, 6)
['order_id', 'date', 'region', 'product', 'quantity', 'revenue']
order_id    int64
date        object
region      object
product     object
quantity    int64
revenue     float64
```

dtype: object

Common student mistakes to watch for:

- Wrong relative path to the CSV (a direct callback to Module 12 Exercise 1) — raises `FileNotFoundError`; run `pwd`/confirm the file's actual location the same way as always.
- Calling `.shape()` with parentheses, expecting it to behave like `.head()` — raises `TypeError: 'tuple' object is not callable`, since `.shape` already *is* the tuple, not a function that produces one; a good, concrete illustration of the attribute-vs-method distinction just introduced.
- Misreading object dtype as meaning “some kind of generic/broken data” rather than specifically “text” — worth restating plainly, since the word choice is genuinely unintuitive.

Check for understanding: “If this dataset had 5,000 rows, why does `.head()` only show 5?” (`.head()`'s whole purpose is a quick, readable preview — printing all 5,000 rows would be unusable as a sanity check; get a student to state that `.head(10)` or any other number can be passed explicitly to change the count, previewing that many pandas methods accept optional arguments like this.)

6.3 Exercise 2 — Filter by Region (0:12–0:20, 8 min)

Teaching goal: The single most important mental model in this entire lab: `df[df['region'] == 'South']` is not a “search” — it’s built from a column of `True/False` values, and the outer brackets keep only the `True` rows. Get this right and every remaining filtering exercise this semester makes sense; get it wrong and pandas filtering stays permanently mysterious.

Say to the class:

“This next line looks like magic the first time you see it, so we’re going to take it apart in two separate steps before running it as one line. This is the single most important idea in today’s whole lab.”

Live-code this in two deliberate stages:

```
# --- Exercise 2, stage 1: see the boolean column itself ---
print(df['region'] == 'South')
```

Run stage 1 and look at the output together — this is the step most guides skip, and it’s the one that actually explains everything:

```
0      False
1       True
2      False
3      False
4      False
...
4999   False
Name: region, Length: 5000, dtype: bool
```

Say explicitly: “`df['region'] == 'South'` produces a brand-new column — not a filtered table, not a search result — a column of `True/False`, one value per row, the same length as the whole DataFrame, saying whether *that specific row* matches the condition.”

Now stage 2 — use that boolean column to actually filter:

```
# --- Exercise 2, stage 2 ---
south = df[df['region'] == 'South']
print(len(south))
print(south.head())
```

Line-by-line explanation:

- `df[df['region'] == 'South']` — read the **outer** brackets as “keep only the rows where this is `True`.” The inner expression, `df['region'] == 'South'`, is exactly stage 1’s boolean column — say explicitly: **this is the same boolean column from a moment ago, now**

placed inside the outer `df[...]` brackets to actually perform the filtering. This two-step mental model — build a True/False column, then use it to select rows — is what every filter this semester will be built from, however complex it eventually gets.

- `south = df[...]` — the result is a **new DataFrame**, `south`, containing only the rows that were True — a genuinely separate object from `df`, not a live “view” that changes if `df` changes later (worth a brief, honest caveat: pandas’ exact internal behavior here — view vs. copy — is more nuanced than this lab needs to cover, and will resurface as a real gotcha in Exercise 5’s common mistakes).
- `len(south)` — Module 06’s familiar `len()`, here reporting the row count of the filtered result.
- `south.head()` — confirms visually that every row shown genuinely has 'South' in the region column.

Run it. Expected output (row count will vary by dataset — verified against the synthetic sample, 1352 of 5000 rows matched 'South'):

1352

	order_id	date	region	product	quantity	revenue
1	100002	2024-05-27	South	Widget B	11	549.89
9	100010	2024-04-25	South	Widget A	11	164.89
10	100011	2024-03-10	South	Gadget D	13	259.87
14	100015	2024-07-10	South	Gizmo E	14	209.86
21	100022	2024-01-01	South	Widget B	13	649.87

Point out explicitly: the row index numbers (1, 9, 10, 14, 21, ...) are **not** 0, 1, 2, 3, 4 — they’re the *original* index values from `df`, carried over into the filtered result. This is worth flagging now, briefly, since Exercise 3 makes the exact same observation about a different filter, and noticing the pattern twice cements it.

Common student mistakes to watch for:

- Using a single `=` instead of `==` — a `SyntaxError`, since `=` alone is assignment, not comparison, exactly the Module 05 lesson resurfacing in a pandas context.
- Skipping straight to the one-line filtered version without seeing the intermediate boolean column — if a student seems to be pattern-matching the syntax without understanding *why* it works, walk them back through stage 1 individually rather than moving on.
- Assuming `south` will automatically update if `df` changes later in the script — worth a brief, honest note that this isn’t reliably true and shouldn’t be depended on; if fresh-filtered data is needed after `df` changes, re-run the filter.

Check for understanding: “If I ran `df['region'] == 'West'` right now, what would print, and how is it different from `df[df['region'] == 'West']`?” (The first is the raw True/False column itself, same length as `df`; the second uses that column to actually filter down to a smaller DataFrame containing only the True rows. Getting a student to state both halves confirms the two-step mental model actually landed.)

6.4 Exercise 3 — Top 5 by Revenue (0:20–0:27, 7 min)

Teaching goal: `.sort_values()` and `.head()` chained together — a very common “top N” pattern — plus a first, gentle introduction to the fact that a DataFrame’s row index doesn’t have to be 0, 1, 2, ... in order.

Say to the class:

“A genuinely common real request: ‘show me the top 5.’ One chained line.”

Live-code this:

```
# --- Exercise 3 ---
top5 = df.sort_values('revenue', ascending=False).head(5)
print(top5)
```

Line-by-line explanation:

- `df.sort_values('revenue', ascending=False)` — sorts the **entire** DataFrame by the revenue column; `ascending=False` means highest-first (descending) — the default, if omitted, is ascending (lowest-first), so this argument is doing real, necessary work here, not just being explicit for style.
- `.head(5)` — **chained directly onto the sort**, with no intermediate variable — say explicitly: `.sort_values(...)` returns a full, sorted DataFrame, and `.head(5)` is then called on *that result*, immediately, in the same line. This chaining style (`.method1().method2()`) is extremely common in pandas and worth naming as a pattern students will see constantly: each method returns something you can immediately call another method on.
- `top5 = ...` — stores the final, chained result.

Run it. Expected output (verified against the synthetic sample — actual rows/values will differ with real course data, but every result should have 20 in the quantity column at this dataset’s price points, since these are the maximum-value transactions):

	order_id	date	region	product	quantity	revenue
4365	104366	2024-07-06	North	Gizmo E	20	1999.8
1913	101914	2024-01-27	South	Gadget D	20	1999.8
3236	103237	2024-06-17	West	Doohickey F	20	1999.8
1857	101858	2024-02-05	South	Doohickey F	20	1999.8
637	100638	2024-02-28	East	Gadget C	20	1999.8

Point out explicitly, matching the lab page’s own required observation: the leftmost index column (4365, 1913, 3236, ...) is **not** 0, 1, 2, 3, 4 — it’s each row’s *original* position from the unsorted df, carried along through the sort. Say plainly: **sorting rearranges row order, but doesn’t renumber the index** — if a clean 0-through-4 index were needed (for example, to later access “the 3rd-highest row” by position), an explicit `.reset_index()` call would be required, which this exercise deliberately doesn’t ask for, specifically so students notice the index *isn’t* automatically

reset.

Common student mistakes to watch for:

- Forgetting `ascending=False` — silently produces the *lowest* 5 values instead of the highest, with no error at all; a good “does this output actually answer the question asked” sanity check, since a revenue column full of tiny numbers should look obviously wrong against “top 5.”
- Being confused or alarmed by the non-sequential index, assuming something went wrong — reassure explicitly that this is expected, correct behavior, not a bug.

Check for understanding: “If two rows had the *exact same* revenue value, how would `.sort_values()` decide which one appears first?” (Not something students need to derive precisely, but a good prompt to introduce the idea that ties are broken by original row order by default, unless a secondary sort column is specified — worth a one-sentence mention that `.sort_values()` accepts a list of columns for exactly this kind of tie-breaking, without demonstrating it live unless time allows.)

6.5 Exercise 4 — Two-Condition Filter (0:27–0:36, 9 min)

Teaching goal: Combine two conditions in a pandas filter — and confront, directly and deliberately, the fact that this requires `&` (not `and`) *and* parentheses around each condition, which is a genuinely confusing, easy-to-get-wrong piece of syntax worth a real live demonstration of both failure modes.

Say to the class:

“Combining two conditions in pandas looks similar to Module 5’s `and`, but it is not the same syntax, and using the wrong one produces a genuinely confusing error. I’m going to show you both ways this breaks, on purpose, before showing you the version that works.”

Live-code the first broken version — using Python’s `and` instead of `&`:

```
# --- Exercise 4, broken attempt 1 ---
big_south = df[df['region']=='South' and df['revenue']>1000]
```

Run it. It raises:

ValueError: The truth value of a Series is ambiguous.
Use `a.empty()`, `a.bool()`, `a.item()`, `a.any()` or `a.all()`.

Explain why, plainly: Python’s `and` is designed to combine two single True/False values — but `df['region']=='South'` isn’t a single value, it’s an entire *column* of thousands of True/False values (Exercise 2’s boolean column). `and` doesn’t know how to meaningfully combine two whole columns at once, so it raises this error rather than guessing. This is worth stating explicitly: **`and/or` are for single booleans; `&|` are pandas’ element-by-element equivalents, built specifically to combine two boolean *columns*, checking each row’s pair of values independently.**

Now the second broken version — using `&` but without parentheses:

```
# --- Exercise 4, broken attempt 2 ---
big_south = df[df['region']=='South' & df['revenue']>1000]
```

Run it. It raises:

TypeError: Cannot perform 'rand_' with a dtyped [float64] array and scalar of type [bool]

Explain why, plainly: `&` has *higher* operator precedence than `==` and `>` in Python — the opposite of what a reader would naturally assume — so without parentheses, this actually gets evaluated as `df['region'] == ('South' & df['revenue']) > 1000`, attempting to `&`-combine the string 'South' with a whole numeric column before the comparisons even happen, which makes no sense and produces this cryptic type error. **State the takeaway plainly, since precision here matters more than intuition:** every condition in a pandas multi-condition filter needs its own parentheses, every time, with no exceptions — this isn’t optional style, it’s required by how Python evaluates the expression.

Now the working version:

```
# --- Exercise 4 ---
big_south = df[(df['region']=='South') & (df['revenue']>1000)]
print(len(big_south))
print(big_south.head())
```

Run it. Expected output (verified against the synthetic sample):

116

	order_id	date	region	product	quantity	revenue
56	100057	2024-05-28	South	Gizmo E	15	1499.85
71	100072	2024-07-09	South	Widget B	20	1999.80
480	100481	2024-06-10	South	Widget B	11	1099.89
499	100500	2024-03-08	South	Gizmo E	17	1699.83
506	100507	2024-04-22	South	Widget A	12	1199.88

Point out explicitly, matching the lab page’s own required observation: 116 is smaller than either single-condition filter alone would be (Exercise 2’s South-only filter had 1352 rows) — say plainly: **& narrows the result; combining two conditions can only keep the same number of rows or fewer than either condition alone, never more.**

Common student mistakes to watch for:

- Both broken versions above — worth having students actually type and run both themselves, not just watch, since the specific error text is exactly what they’ll encounter independently later this semester if they forget this lesson.
- Using | (or) when & (and) was intended, or vice versa — syntactically valid either way, so no error at all, just a silently different (and possibly much larger or much smaller) result set; a good “does this row count look plausible” check.

Check for understanding: “Without running it, would `df[(df['region']=='South') | (df['revenue']>1000)]` return more or fewer rows than the `&` version above?” (More — | (or) keeps a row if *either* condition is true, a strictly looser filter than `&`’s “both required” — get a student to reason through this from the `&/|` definitions rather than just guessing.)

6.6 Exercise 5 — A New Column (0:36–0:44, 8 min)

Teaching goal: Create a new column with a single vectorized assignment — no loop, no `.append()` — directly contrasting with Module 12’s row-by-row CSV processing, and computing a summary statistic with `.mean()`.

Say to the class:

“In Module 12, computing a new value per row meant a loop and an accumulator. Today, one line, no loop at all — pandas applies the calculation to every row simultaneously.”

Live-code this:

```
# --- Exercise 5 ---
df['per_unit'] = df['revenue'] / df['quantity']
print(df.head())
print(df['per_unit'].mean())
```

Line-by-line explanation:

- `df['per_unit'] = df['revenue'] / df['quantity']` — this single line computes revenue / quantity for **every row at once**, and stores the result as a brand-new column called `per_unit`. Say explicitly, since it’s genuinely different from every prior module’s approach: **there’s no loop here at all** — `df['revenue']` and `df['quantity']` are both whole columns, and dividing one column by another produces a third column, row by row, automatically. This is called **vectorized** computation, and it’s both dramatically shorter to write and, on large datasets, dramatically faster to run than an equivalent hand-written loop.
- `df['per_unit'].mean()` — `.mean()` is a built-in aggregation method, computing the average of an entire column in one call — parallel to, but far more concise than, Module 10’s manual `sum(list) / len(list)`.

Run it. Expected output (verified against the synthetic sample — `.head()` output abbreviated here to the new column’s presence, mean value shown exactly):

```
order_id  ... quantity  revenue  per_unit
0    100001  ...      2    19.98     9.99
1    100002  ...     11   549.89    49.99
...
37.023
```

Common student mistakes to watch for:

- Trying to create the new column with a loop and `.append()`-style thinking carried over from Module 10/12, rather than the direct vectorized assignment — not wrong in the sense of being impossible (pandas *can* be looped over row by row, with `.iterrows()`, not covered in this lab), but far slower and more verbose than necessary; if a student starts writing a loop

here, it's worth asking “do you remember Exercise 2's boolean column — did that need a loop either?” to nudge toward the vectorized instinct.

- **Attempting the assignment on a filtered copy instead of the original df** — e.g., accidentally writing `south['per_unit'] = ...` instead of `df['per_unit'] = ...` — this can raise a real pandas warning, `SettingWithCopyWarning`, because `south` (from Exercise 2) is a filtered subset whose relationship to the original `df` is ambiguous to pandas. This is a genuine, very commonly-encountered pandas gotcha professionally, not just a classroom simplification — worth demonstrating live if you have the extra minute, since students will hit this warning again later in the course and should recognize it rather than be alarmed by it.

Check for understanding: “If `quantity` were 0 for some row, what would happen to that row's `per_unit` value?” (Division by zero in pandas produces `inf` — infinity — rather than crashing the whole program the way plain Python's `1/0` would raise `ZeroDivisionError`; a genuinely useful, real distinction worth surfacing even briefly, since it means a 0 in a denominator column can silently produce a technically-valid-looking but meaningless result rather than an obvious crash.)

6.7 Exercise 6 — Unique Values (0:44–0:51, 7 min)

Teaching goal: `.unique()` (what distinct values exist) and `.value_counts()` (how often each one appears) — two of the most commonly reached-for exploratory methods in real data work, and a natural pairing worth learning together.

Say to the class:

“Two quick, extremely common questions about any column: what values does it actually contain, and how many of each?”

Live-code this:

```
# --- Exercise 6 ---
print(df['region'].unique())
print(df['region'].value_counts())
```

Line-by-line explanation:

- `df['region'].unique()` — returns each **distinct** value in the column exactly once, in the order first encountered (not sorted) — a quick way to answer “what are all the possible categories here,” especially useful on an unfamiliar dataset before deciding how to filter or group it.
- `df['region'].value_counts()` — returns a count of how many rows contain each distinct value, automatically **sorted from most to least common** — say explicitly this default sort order (most common first) is often exactly what you want when scanning for the dominant categories in a column.

Run it. Expected output (verified against the synthetic sample):

```
['East' 'South' 'West' 'North']
region
North    1442
South    1352
East     1121
West     1085
Name: count, dtype: int64
```

Point out explicitly: the row counts here sum to 5000 — the full dataset — and if a student’s real course data has any missing/null region values, those often show up separately or are excluded by default, worth a brief mention (`.value_counts(dropna=False)` reveals nulls explicitly, if relevant to your specific dataset) without a full detour into missing-data handling, which Module 14 covers properly.

Common student mistakes to watch for:

- Confusing `.unique()` (a plain array/list of distinct values, unordered by frequency) with

`.value_counts()` (a count *per* value, ordered by frequency) — a quick side-by-side glance at both outputs' shape (a flat list vs. a value-with-a-number-next-to-it table) usually resolves the confusion fast.

- Calling `.unique()` or `.value_counts()` on the whole `df` instead of a single column (`df.unique()` rather than `df['region'].unique()`) — raises `AttributeError`, since these methods exist on a single column (a pandas **Series**), not on a whole multi-column `DataFrame`; a good moment to introduce, briefly, that `df['region']` alone (no further indexing) is a *Series*, one column pulled out on its own, and that some methods are *Series*-only.

Check for understanding: “How would you find out how many *different products* are sold, without listing every single product name?” (`df['product'].nunique()` — the count of unique values directly, rather than `len(df['product'].unique())`, though both work; if `nunique()` hasn't been introduced, accepting the `len(...unique())` combination as a correct, reasoned answer is completely fine — the point is recognizing the question maps to `.unique()`-adjacent tools at all.)

6.8 Exercise 7 — Select Columns (0:51–0:57, 6 min)

Teaching goal: Create a new DataFrame containing only a subset of columns — a quick, common operation for narrowing focus (and reducing memory) before deeper analysis.

Say to the class:

“Sometimes you don’t want every column, just a few — here’s the syntax for pulling out exactly the ones you need.”

Live-code this:

```
# --- Exercise 7 ---
subset = df[['region', 'product', 'revenue']]
print(subset.shape)
```

Line-by-line explanation:

- `df[['region', 'product', 'revenue']]` — note the **double square brackets**: the outer `[...]` is the same “select something from `df`” syntax as every filter so far, and the inner `[...]` is an ordinary Python **list** of column names. Say explicitly, since this is a genuinely easy detail to drop by accident: **a single set of brackets with one column name (`df['region']`) gives back a Series (one column); double brackets with a list of names (`df[['region', 'product']]`) gives back a DataFrame (a table, even if it only has one column in the list).** This distinction — one bracket vs. two — trips up nearly everyone at least once.
- `subset.shape` — same (rows, columns) attribute from Exercise 1, now confirming the row count is unchanged (still every row) while the column count has dropped to exactly 3.

Run it. Expected output:

`(5000, 3)`

Common student mistakes to watch for:

- Writing `df['region', 'product', 'revenue']` (single brackets, comma-separated names, no inner list) — raises `KeyError`, since pandas interprets this as looking for one single column literally named `('region', 'product', 'revenue')` (a tuple), which doesn’t exist. This is worth demonstrating live specifically because the error message can look confusingly like *any* of the individual names might be the problem, when the real issue is the missing inner brackets.
- Assuming `subset` shares live data with `df` such that modifying one affects the other — same honest caveat as Exercise 2’s filtered result; not something to rely on.

Check for understanding: “What would `df[['revenue']]` — a single column name, but still inside double brackets — return: a Series or a DataFrame, and how is that different from `df['revenue']` alone?” (A DataFrame, with exactly one column — this is precisely the distinction

tonight’s reflection question 2 asks students to articulate, so rehearsing it here, out loud, previews that exact question directly.)

6.9 Exercise 8 — Save (0:57–1:02, 5 min)

Teaching goal: Write a DataFrame back out to CSV with `.to_csv()` — the “Load” step of this lab’s ETL framing, and a direct pandas replacement for Module 12’s entire manual `csv.writer` block.

Say to the class:

“Last step: save your work back to a file. One line, replacing Module 12’s entire manual write-loop.”

Live-code this:

```
# --- Exercise 8 ---  
df.to_csv('data/explored.csv', index=False)
```

Line-by-line explanation:

- `df.to_csv('data/explored.csv', index=False)` — writes the *entire current state* of `df` (including Exercise 5’s new `per_unit` column, since it was added directly to `df` itself) to a new CSV file.
- `index=False` — **this argument matters, and is worth explaining rather than treated as boilerplate to memorize:** by default, `.to_csv()` would write the DataFrame’s row index (the 0, 1, 2, ... — or, after Exercise 3’s sort, the *non-sequential* — numbers on the left) as its own extra column in the output file. `index=False` skips that, writing only the actual data columns. Ask the room: “based on what we saw in Exercise 3, why might writing the index out to a file sometimes be actively misleading rather than just extra?” (Because after a sort, the index numbers aren’t sequential or meaningful to someone opening the CSV fresh — they’d look like a mysterious, gappy ID column rather than what they actually are.)

Verify: open `data/explored.csv` in a text editor or spreadsheet application and confirm the header row and a few sample rows look correct, including the `per_unit` column from Exercise 5.

Common student mistakes to watch for:

- Forgetting `index=False` — not an error, just an extra, likely-confusing unnamed column in the output file; open the file together and point at the artifact directly if it happens.
- Running Exercise 8 *before* Exercise 5 in their actual script (out of the stated exercise order) and being confused that `per_unit` doesn’t appear in `explored.csv` — a good reminder that a script runs top to bottom, and `.to_csv()` only saves whatever `df` currently *contains* at the moment it’s called, not anything added afterward.

Check for understanding: “If you reran this entire script tomorrow with the same source CSV, would `explored.csv` change?” (No — same input, same deterministic transformations, same output; a good sanity check that nothing in today’s pipeline depends on anything outside the script itself, unlike, say, Module 12 Exercise 8’s timestamp-dependent log.)

6.10 Stretch 1 & 2 Preview (1:02–1:10, as time allows)

Frame both as previews rather than full live-coded content — Stretch 1 explicitly uses `groupby`, which the lab page itself flags as a Module 15 concept, ahead of this lab's normal sequence:

Stretch 1 — Top 3 products by average per-unit price in the South:

```
south = df[df['region'] == 'South']
print(south.groupby('product')['per_unit'].mean().sort_values(ascending=False).head(3))
```

If you demo this, one sentence of framing is enough: “`groupby('product')` splits the South data into one group per product name; `['per_unit'].mean()` then computes the average per-unit price *within each group separately*; the rest is Exercise 3’s familiar sort-and-head pattern. Don’t worry about fully understanding `groupby` today — Module 15 covers it properly; this is just a preview that today’s filtering and column-creation skills combine directly with tools you’ll learn soon.”

Stretch 2 — What fraction of revenue comes from orders above \$500?

```
big_orders = df[df['revenue'] > 500]
fraction = big_orders['revenue'].sum() / df['revenue'].sum() * 100
print(f"{fraction:.1f}%")
```

This one is fully within today’s toolkit — Exercise 2’s filtering pattern, plus `.sum()` (parallel to Exercise 5’s `.mean()`), plus an f-string format spec from Module 03. Worth doing live if time allows, since — unlike Stretch 1 — it requires nothing beyond what’s already been taught this session. (Verified against the synthetic sample: 59.7%.)

7 Wrap-Up (last ~5 minutes)

Review the reflection questions out loud:

1. *Explain a boolean filter in plain English* — a strong answer states, unprompted, the two-step model from Exercise 2: build a column of True/False, then keep only the True rows — not just “it searches for matching rows,” which skips the actual mechanism.
2. *df["revenue"] vs. df[["revenue"]]* — the single-vs-double-bracket distinction from Exercise 7: a Series versus a one-column DataFrame. Push for *when it matters*: many DataFrame-only methods (like `.to_csv()` in some contexts, or later chaining onto another DataFrame method) require a DataFrame, not a Series — a good answer names at least one concrete situation where the distinction has real consequences, not just “they’re technically different.”
3. *The single most compelling reason to choose pandas over Excel* — no wrong answer, but push past “it’s more powerful” for something specific: scale (thousands to millions of rows without manual scrolling), reproducibility (the exact same script re-run on updated data, versus re-doing manual Excel steps), or being embeddable in a larger automated pipeline are all strong, concrete answers.

Review the submission checklist together:

- ☐ File is named `explore.py` (or `explore.ipynb`)
- ☐ Contains Exercises 1–8, each clearly separated
- ☐ Runs top to bottom with no errors
- ☐ Pushed to GitHub inside a `module13/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 14: “Today assumed the data was already clean — no missing values, no weird formatting. Next module tackles the reality that real data almost never starts that clean: missing values, duplicate rows, and descriptive statistics to understand a dataset’s overall shape before analyzing it further.”

8 Appendix A — Full Answer Key (`explore.py`)

```
# explore.py
# ISM2411 Module 13 Lab – First DataFrame: Retail Sales Explorer

import pandas as pd

# --- Exercise 1 ---
df = pd.read_csv('data/retail_sales.csv')
print(df.head())
```

```

print(df.shape)
print(df.columns.tolist())
print(df.dtypes)

# --- Exercise 2 ---
south = df[df['region'] == 'South']
print(len(south))
print(south.head())

# --- Exercise 3 ---
top5 = df.sort_values('revenue', ascending=False).head(5)
print(top5)

# --- Exercise 4 ---
big_south = df[(df['region']=='South') & (df['revenue']>1000)]
print(len(big_south))
print(big_south.head())

# --- Exercise 5 ---
df['per_unit'] = df['revenue'] / df['quantity']
print(df.head())
print(df['per_unit'].mean())

# --- Exercise 6 ---
print(df['region'].unique())
print(df['region'].value_counts())

# --- Exercise 7 ---
subset = df[['region', 'product', 'revenue']]
print(subset.shape)

# --- Exercise 8 ---
df.to_csv('data/explored.csv', index=False)

```

Stretch 1 (groupby preview):

```

south = df[df['region'] == 'South']
print(south.groupby('product')['per_unit'].mean().sort_values(ascending=False).head(3))

```

Stretch 2 (% of revenue from big orders):

```
big_orders = df[df['revenue'] > 500]
fraction = big_orders['revenue'].sum() / df['revenue'].sum() * 100
print(f"{fraction:.1f}%")
```

9 Appendix B — Reproducible Synthetic Dataset (for instructor testing)

If your course's real `retail_sales.csv` isn't yet available for pre-class testing, this generator produces a structurally equivalent, reproducible dataset (fixed random seed) matching the lab page's expected (5000, 6) shape. **Use the actual course-provided file with students** — this is for your own rehearsal only.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 5000

regions = ["North", "South", "East", "West"]
products = ["Widget A", "Widget B", "Gadget C", "Gadget D", "Gizmo E", "Doohickey F"]

region_arr = rng.choice(regions, size=n, p=[0.28, 0.27, 0.23, 0.22])
product_arr = rng.choice(products, size=n)
quantity_arr = rng.integers(1, 21, size=n)
unit_price = rng.choice([9.99, 14.99, 19.99, 24.99, 49.99, 99.99], size=n)
revenue_arr = np.round(quantity_arr * unit_price, 2)
dates = pd.date_range("2024-01-01", periods=200, freq="D")
date_arr = rng.choice(dates, size=n)
order_id = np.arange(100001, 100001 + n)

df = pd.DataFrame({
    "order_id": order_id,
    "date": pd.to_datetime(date_arr).strftime("%Y-%m-%d"),
    "region": region_arr,
    "product": product_arr,
    "quantity": quantity_arr,
    "revenue": revenue_arr,
})
df.to_csv("data/retail_sales.csv", index=False)
```

All expected output shown throughout this guide (row counts, means, top-5 values, the Stretch 2 percentage) was computed and verified against this exact generator's output.

10 Appendix C — Extra Practice (only if the class finishes early)

Eight required exercises fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a third single-condition filter, then compare. Have students filter for `df[df['product'] == 'Gizmo E']`, print the count, then compare it against `df['product'].value_counts()`'s own reported count for 'Gizmo E' — these should match exactly, a good independent cross-check that both the filter and `.value_counts()` are behaving as understood.

Extra — one more two-condition filter, different columns. Have students build `df[(df['product'] == 'Widget A') & (df['quantity'] >= 10)]`, print the count and `.head()`, and state in one sentence what real business question this filter answers (e.g., “large Widget A orders” — worth pushing for a plausible business framing, not just a syntactically correct filter).